

GPU 并行图着色算法实现与分析

1. 背景

在基于 AVBD (Augmented Vertex Block Descent) 的刚体仿真中，约束求解采用 Gauss-Seidel 迭代。CPU 上可以按顺序逐 body 更新，但 GPU 并行化要求同一轮内更新的 body 之间不存在共享约束。

将约束关系建模为**碰撞图** (body 为顶点，manifold 为边)，对该图着色后，**同色顶点互不相邻**，可以并行求解。颜色数越少，并行度越高 (每次迭代需要的 kernel dispatch 数 = 颜色数)。

本文实现并比较了 Vivace 论文 (SIGGRAPH Asia 2016, Fratarcangeli et al.) 中的四种 GPU 并行图着色算法。同时提供了一个 Python 演示脚本 ([doc/graph_coloring_demo.py](#))，可以在一个 10 节点的小图上逐步输出四种算法的完整执行过程。

运行方法：`python doc/graph_coloring_demo.py`

2. 数据来源

碰撞图来自 GPU narrowphase 输出的 per-body 邻接表：

- `vtx_counts[body]`：邻居数量 (该 body 与多少个其他 body 碰撞)
- `vtx_table[body * 8 + slot].other_body`：邻居 body 索引
- 最大邻居数 `VERTEX_TABLE_MAX_NEIGHBORS = 8`

测试场景：**金字塔堆叠** (Pyramid)，137 个刚体 OBB，稳态约 270–284 个 manifold。

3. 四种算法详解

3.1 Brooks-Vizing 随机着色 (Vivace)

出处：Grable & Panconesi 2000，被 Vivace 论文选为其核心着色策略。

核心思想：每个顶点维护一个“调色盘” (palette)，即一个可选颜色的集合。每轮迭代三步操作，每步各用一个 GPU kernel 实现。

初始化

- 计算缩减因子 `s = min_degree(G)` (图中最小的非零度，论文推荐)
- 每个顶点 `v` 分配调色盘 `P_v = {0, 1, ..., floor(deg(v)/s)}`
- 度越高的顶点，调色盘越大

直觉：调色盘限制了每个顶点可以选的颜色范围，避免用太多颜色。`s` 的作用是把调色盘缩小到比度数小，减少总颜色数。

每轮迭代三步

(a) Tentative Coloring (试探着色)

每个未着色顶点从自己的调色盘中**随机选一个颜色**。这一步是完全并行的——每个顶点独立选色。

(b) Conflict Resolution (冲突解决)

检查每个顶点的试探颜色是否与某个邻居的试探颜色相同：

- 如果没有邻居选了相同颜色 → 确认着色，并从所有邻居的调色盘中**移除**该颜色
- 如果有邻居选了相同颜色 → 使用**匈牙利启发式** (Hungarian heuristic) 打破平局：索引更大的顶点保留着色，索引小的放弃

为什么要移除颜色：如果邻居 u 已经被确认着色为颜色 c ，那么 v 以后也不能用颜色 c ，所以从 P_v 中移除 c 。

(c) Feed the Hungry (喂饱饥饿者)

有些顶点的调色盘可能因为多次移除而变空了 ("饥饿")。此时给所有饥饿顶点的调色盘加入**同一个全局新颜色**。

关键细节：所有饥饿顶点共享同一个新颜色号，而不是每个饥饿顶点各自获取独立的颜色号。这对控制总颜色数至关重要。

具体示例 (Python 脚本输出)

以一个 10 节点碰撞图为例 (body 4 度=6 最高，body 0/6/9 度=2 最低， $s=2$)：

第 1 轮：

试探着色：body0-→0, body1-→0, body2-→1, body3-→0, body4-→1, ...

冲突解决：body0 和 body1 都选了 0, $0 < 1$ 所以 body0 放弃

body4 的颜色 1 无冲突 → 确认着色

body6, body8, body9 无冲突 → 确认着色

feed-the-hungry: body5 和 body7 的调色盘被移除为空 → 加入新颜色 4

第 2 轮：

body0 选 1, body2 选 0, body3 选 2, body5 选 4, body7 选 4

body0/2/3/5/7 全部无冲突 → 确认

body1 冲突放弃，调色盘空 → 加入新颜色 5

第 3 轮：

body1 选 5 → 确认

结果：5 种颜色，3 轮

特点：

- **优势**：收敛快 (轮次少)，因为每轮可以着色很多顶点
- **劣势**：颜色数略多 (因为调色盘机制和新颜色追加)，但远好于 Luby
- **复杂度**： $O(\log n)$ 轮 (理论)，每轮 3 个 kernel

3.2 Luby MIS (最大独立集) 着色

出处：Luby 1985，经典的并行图算法。

核心思想： 每轮用随机权重找一个**"最大独立集"**（MIS）——一组互不相邻的顶点。独立集中所有顶点赋**同一颜色**，然后移除这些顶点，进入下一轮用下一个颜色。

每轮操作

1. **赋随机权重：** 每个未着色顶点获得一个随机整数
2. **找局部极大值：** 如果顶点的权重比它的所有未着色邻居都大（相等时索引大的优先），则它是**"局部极大值"**
3. **构成独立集：** 所有局部极大值构成一个独立集（它们互不相邻，因为如果两个相邻顶点都是局部极大值，则矛盾）
4. **赋同一颜色：** 独立集内所有顶点赋当前轮次对应的颜色

具体示例

第 1 轮 (颜色 0):
随机权重: {0:654, 1:114, 2:25, 3:759, 4:281, 5:250, 6:228, 7:142, 8:754, 9:104}
局部极大值: body3(759) 和 body8(754) – 它们互不相邻
body3→颜色0, body8→颜色0

第 2 轮 (颜色 1):
body2(913) 和 body6(604) 是局部极大值
body2→颜色1, body6→颜色1

第 3 轮 (颜色 2): body7, body9 → 颜色2

第 4 轮 (颜色 3): body1, body5 → 颜色3

第 5 轮 (颜色 4): body0, body4 → 颜色4

结果: 5 种颜色, 5 轮

特点：

- **优势：** 实现最简单，每轮只需 2 个 kernel

• **劣势：** 颜色数 = 轮次数（因为每轮一种颜色），颜色较多

• **为什么颜色多：** 同一独立集的顶点互不相邻，完全可以赋不同颜色来复用旧颜色，但 Luby 不这样做

3.3 Jones-Plassmann (JP) 着色

出处：Jones & Plassmann 1993，对 Luby 的关键改进。

核心思想： 和 Luby 一样用随机权重找独立集，但每个独立集顶点不是赋同一颜色，而是赋**最小可用色**——即不与任何已着色邻居冲突的最小颜色号。

与 Luby 的关键区别

	Luby	JP
独立集内着色	全赋同一颜色	各自赋最小可用色

	Luby	JP
颜色复用	不复用	积极复用
颜色数	= 轮次数	<< 轮次数

具体示例

第 1 轮：

独立集：{body3, body8} (与 Luby 相同)

body3：无已着色邻居 → 最小可用色 = 0

body8：无已着色邻居 → 最小可用色 = 0

(注意：body3 和 body8 不相邻，可以用同一颜色！)

第 2 轮：

独立集：{body2, body6}

body2：无已着色邻居 → 颜色 0

body6：邻居 body3 已用颜色 0 → 最小可用色 = 1

第 3 轮：

独立集：{body7, body9}

body7：邻居已用 {0, 1} → 颜色 2

body9：邻居已用 {0} → 颜色 1

第 4 轮：

独立集：{body1, body5}

body1：邻居已用 {0} → 颜色 1

body5：邻居已用 {0, 1} → 颜色 2

第 5 轮：

独立集：{body0, body4}

body0：邻居已用 {0, 1} → 颜色 2

body4：邻居已用 {0, 1, 2} → 颜色 3

结果：4 种颜色，5 轮

特点：

- **优势**：颜色数显著少于 Luby (4 vs 5)，因为积极复用颜色
- **劣势**：需要扫描邻居计算最小可用色，增加一点计算量
- **推荐**：JP 是实际使用的最佳选择，平衡了颜色质量和速度

3.4 LDF (Largest-Degree-First) 着色

出处：Welsh & Powell 1967 的并行版本。

核心思想：和 JP 类似，也赋最小可用色，但优先级不是随机权重，而是**残余度数**——未着色邻居越多的顶点优先着色。每轮后更新残余度。

每轮操作

1. 找**度最大的局部极大值**独立集 (残余度最高且比所有未着色邻居都高的顶点)
2. 每个独立集顶点赋最小可用色
3. 更新残余度：对每个未着色顶点，重新计算它有多少未着色邻居

具体示例

第 1 轮：
残余度：{0:2, 1:4, 2:3, 3:5, 4:6, 5:4, 6:2, 7:4, 8:4, 9:2}
度最大：body4(度=6) – 唯一的局部极大值
body4 → 颜色 0

第 2 轮：
更新残余度：body3 从 5 降到 4 (去掉了 body4)
度最大：body3(4), body8(3) – 互不相邻
body3 → 颜色 1 (邻居用了 {0})
body8 → 颜色 1 (邻居用了 {0})

第 3 轮：
度最大：body5(2), body7(1)
body5 → 颜色 2 (邻居用了 {0, 1})
body7 → 颜色 2 (邻居用了 {0, 1})

第 4 轮：
body1, body6, body9
body1 → 颜色 2, body6 → 颜色 0, body9 → 颜色 0

第 5 轮：
body0 → 颜色 0, body2 → 颜色 1

结果：3 种颜色 · 5 轮

特点：

- **优势**：颜色数最少 (接近色数下界 $\chi(G)$) · 因为高约束度顶点优先处理
- **劣势**：轮次最多 (因为每轮的独立集更小——高度顶点少) · 需要额外的度数更新 kernel
- **适用场景**：对颜色数要求极严格的场景

4. GPU 实现要点

所有四种算法都运行在 GPU 上，核心数据结构直接复用 narrowphase 输出的 per-body 邻接表：

vtx_counts_dev[body]

→ 该 body 的邻居数

vtx_table_dev[body * 8 + slot].other_body

→ 第 slot 个邻居的 body 索引

GPU 并行度：每个 body 对应一个 CUDA thread。

Vivace 的 **palette** 存储：用 bitmask 表示，每个 body 有 **palette_width** 个 int (每个 int 32 位 = 32 种颜色)，用 **__popc** 统计可用色数量，**__ffs** 快速找到第 **k** 个可用色。

原子操作：

- Vivace 的 conflict resolution 中移除邻居 palette 的颜色：**atomicAnd**
- 所有算法的 uncolored count：**atomicAdd**

5. 实测数据

测试环境：NVIDIA RTX 2060 (sm_75), CUDA 13.0, Windows 10

5.1 典型帧数据 (稳态，~270 manifolds, 137 bodies)

算法	颜色数	轮次数	耗时 (ms)
Vivace	7-8	3-5	0.5-1.4
Luby	5-7	5-7	0.5-1.0
JP	4-5	5-8	0.3-1.2
LDF	4	10-11	0.6-2.0

5.2 分析

颜色数排序 (从少到多)：

```
LDF (4) <= JP (4-5) < Luby (5-7) < Vivace (7-8)
```

- **LDF** 始终保持 4 色，这是该碰撞图的色数下界 (金字塔图的最大团大小为 4，因此 $\chi(G) \geq 4$)
- **JP** 接近最优，通常 4-5 色
- **Luby** 由于同一独立集赋同色，无法复用颜色，典型 6-7 色
- **Vivace** 约 8 色，与论文中类似图结构的结果一致 (论文 Cloth 场景 avg deg 5.92 时 6 色)

轮次数排序 (从少到多)：

```
Vivace (3-5) < Luby/JP (5-8) < LDF (10-11)
```

耗时特征：

- 对于 137 bodies 的小规模场景，各算法耗时差异不大 (都在亚毫秒到 2ms 之间)
- GPU kernel launch 和 D2H 同步是主要开销
- 在 >1K bodies 的大场景中，差异会更明显

5.3 对比论文数据

论文中 Bunny 场景 (12118 particles, avg deg 10.07) 的数据：

算法	颜色数 (论文)	颜色数 (我们)
Vivace	13.7	7-8
Luby	21.1	5-7
JP	11.6	4-5
LDF	10.8	4

我们的颜色数普遍更少，因为碰撞图的平均度 (~4) 远小于论文的 mesh 约束图 (avg deg ~10)。图越稀疏，着色越容易。

6. 算法选择建议

应用场景	推荐算法	理由
实时仿真 (帧间重着色)	JP	颜色少 (并行度高)、速度较快、质量稳定
大规模场景 (> 1K bodies)	JP 或 LDF	颜色质量最重要，决定 solver 并行效率
快速原型验证	Vivace	收敛最快，适合调试
需要最少颜色	LDF	始终最优，但轮次最多

对于 AVBD solver 的 Gauss-Seidel 并行化，**JP** 是最佳平衡：

- 4-5 色意味着 solver 每次迭代只需 4-5 轮 color-group 并行 dispatch
- 着色耗时 < 1ms，相对 solver 迭代 (10次 x N bodies) 可忽略

7. 创新方向

7.1 自适应缩减因子 s 的动态调节

Vivace 的缩减因子 s 目前使用图的最小度，但可以根据图的度分布动态调整：

- 稀疏图 (平均度 < 4) 时增大 s，减小调色盘，减少颜色
- 密集图 (平均度 > 6) 时减小 s，加快收敛

可在每帧开始时用一个轻量 reduction kernel 统计度分布，自动计算最优 s。

7.2 结合 BVH 空间局部性的着色优化

当前着色完全基于图拓扑，忽略了空间信息。QuantBVH 的层级结构天然提供了空间聚类：

- **BVH 子树预着色**：同一 BVH 叶节点的 body 大概率相邻，可以先在子树内贪心着色，再跨子树合并
- **Morton 码排序**：body 按 Z-order 排列后，局部性更好，LDF 的 degree 计算更 cache-friendly

7.3 两阶段着色：broadphase 预着色 + narrowphase 精修

AABB broadphase 产生的碰撞图是 SAT narrowphase 碰撞图的超集：

1. 阶段一：在 broadphase 之后立即着色 (AABB 图，边更多)
2. 阶段二：narrowphase 移除了一些边后，只需对受影响的顶点做增量重着色

优势：broadphase 着色可以和 narrowphase 计算并行（不同 CUDA stream）。

7.4 帧间时间相干性：增量重着色

物理仿真中，相邻帧的碰撞图变化很小（通常只有几条边增删）：

- 保留上一帧着色作为初始状态
- 只对新增边端点和删除边端点进行局部重着色
- 检测冲突的 kernel 只处理变化区域

实现路径：

1. 维护 `prev_vtx_counts` 和 `prev_vtx_table`
2. Diff kernel：找出新增/删除的边，标记受影响的顶点
3. 只对标记顶点运行 JP 着色（其他顶点保持原色）

预期：稳态时仅需 1-2 轮即可完成重着色，耗时降低 5-10x。

7.5 Warp-level 协作着色

当前实现是 1 thread per vertex。对于高度顶点，单线程遍历邻居是瓶颈：

- **Warp-cooperative**：一个 warp（32 threads）协作处理一个顶点
 - 每个 lane 检查不同邻居是否冲突
 - `__ballot_sync` + `__shfl_sync` 实现 warp-level reduction
 - 特别适合 JP/LDF 的“最小可用色”计算

7.6 基于碰撞图结构的混合着色

不同区域的碰撞图密度不同。可以根据 BVH 聚类将 body 分成“密集组”和“稀疏组”，分别用不同算法（密集用 LDF，稀疏用 JP），最后合并。

8. 扩展阅读：CJP 和 CC 算法

论文 "*Efficient Algorithms for Graph Coloring on GPU*" (Pham & Fan, ICPADS 2018) 提出了两种更先进的 GPU 着色算法。Python 演示脚本 `doc/graph_coloring_cjp_cc_demo.py` 逐步展示了它们的执行过程。

8.1 CJP (Counting-based Jones-Plassmann)

CJP 对经典 JP 算法做了关键优化——用**计数器**替代每轮重新赋权重：

1. 只在开头赋一次随机值 `val(v)`
2. 每个顶点维护 `count(v)` = 比自己 `val` 大的未着色邻居数
3. `count(v)=0` 时 `v` 是局部极大值，可以着色
4. `v` 着色后递减邻居的 `count` (`atomicSub`)，若邻居 `count` 变为 0，加入下一轮队列

工作效率 **O(m)**：每条边只被访问常数次，而非每轮都扫描所有边。

GPU 友好：

- `count` 递减用原子操作，天然并行
- 队列用 prefix-sum 构建

- 最小可用色用 256-bit bitmask + `__ffs` 指令

8.2 CC (Conflict Coloring)

CC 进一步提升了 GPU 利用率，核心是“乐观着色 + 冲突修复”：

1. 每轮让**所有**未着色顶点同时选色（最大化并行度！）
2. 允许临时冲突（邻居同色），然后检测冲突
3. 冲突时度小的一方放弃颜色，下轮重试
4. 维护全局 `max_color`，动态增长到合适大小

CC 的 GPU 并行语义（关键）

CC 不是顺序地一个一个处理顶点，而是所有顶点同时操作：

```
Phase 1 -- 并行选色：
    snapshot = copy(colors)           // 先拍摄颜色快照
    for all v in queue IN PARALLEL:   // GPU: 一个 thread 处理一个顶点
        used = {snapshot[nb] for nb in adj[v] if snapshot[nb] >= 0}
        available = {0..max_color-1} - used
        colors[v] = random_choice(available) // 基于快照独立选色

Phase 2 -- 冲突检测：
    for all (v, nb) IN PARALLEL:      // 所有顶点选完后统一检测
        if colors[v] == colors[nb]:
            loser = (v if deg[v] < deg[nb] else nb)
            colors[loser] = -1        // 度小的放弃
```

为什么需要快照：如果没有快照，线程 A 修改 `colors[A]` 后，线程 B 读到的可能是新值也可能是旧值（race condition）。GPU 上所有线程基于同一快照做决策，然后统一写入，最后再统一检测冲突。

这意味着两个邻居可能同时选了相同颜色 -- 因为它们各自看到的快照中对方都是未着色状态。这是 CC 的核心特点：允许冲突存在，然后高效修复。

具体示例（15 body 金字塔）

```
CC 第 1 轮：15 活跃，max_color=4
    所有 15 顶点同时选色（基于初始快照，无人着色）：
        body 0 -> 3, body 1 -> 0, body 2 -> 0, body 3 -> 0, ...
        body 4 -> 1, body 5 -> 1, body 7 -> 1, body 8 -> 3
        body 9 -> 1, body 10 -> 3, body 11 -> 2, body 12 -> 0
    冲突检测：8 对冲突，8 顶点需重着色
    (body0=3 和 body8=3 相邻, body5=1 和 body7=1 相邻 ...)
```

```
CC 第 2 轮：8 活跃，max_color=4
    8 个冲突者基于当前快照重新选色
    冲突：2 对
```

```
CC 第 3 轮：2 活跃，max_color=6
```

剩余 2 顶点无冲突，着色完成

结果：6 种颜色，3 轮

关键洞察：选**随机**未用色（而非最小未用色），使冲突率极低（实测平均 1.67 次/顶点）。

	CJP	CC
颜色质量	接近最优	多约 10%
速度	1.5-2x faster than prior art	2.7-4.3x faster
轮次	20-35	5-7
GPU 利用率	受独立集大小限制	所有顶点参与

对我们 **AVBD solver** 的启示：

- 当前 JP 实现已经接近 CJP 的着色质量
- 若着色成为性能瓶颈（大规模场景），可以考虑替换为 CC
- CC 的 `max_color` 自适应机制比 Vivace 的 `feed-the-hungry` 更优雅

9. 帧间增量着色: CC-Incremental

9.1 核心洞察

CC 天然适合做增量着色，因为它的算法框架本身就是“乐观着色 + 冲突修复”：

- **全量 CC**：所有顶点从 `colors[v] = -1` 开始，随机选色，检测冲突，修复
- **CC-Incremental**：所有顶点从 `colors[v] = prev_frame_colors[v]` 开始，**跳过选色**，直接检测冲突，修复

两者的区别仅在于**初始状态**。CC 的冲突检测/修复逻辑完全不变——同一套 kernel 代码，不同的初始值。

9.2 max_color 机制详解

`max_color` 是 CC 的全局变量，表示当前允许使用的最大颜色数（论文 Section III-B）：

初始化：`max_color = 4`（论文硬编码，作为色数 $\chi(G)$ 的下界估计）

溢出处理：当顶点 `v` 发现 `[0, max_color)` 范围的颜色全被邻居占用：

```
old = atomicAdd(&max_color, 1); // GPU 原子操作, max_color 变为 max_color+1
color[v] = old;                // v 使用新颜色号
```

- 同一轮多个顶点可能同时溢出，各自 `atomicAdd`，各拿不同的新颜色号
- `max_color` 只增不减，2-3 轮后稳定
- 论文实测：K6 完全图从 `max_color=4` 开始，4 轮后增长到 6（正好 $\chi(K6)=6$ ）

CC-Incremental 中的 `max_color`：

```
max_color = prev_frame_max_color; // 直接复用上一帧的最终值
```

因为帧间图结构相似，色数也相似。如果新图更密，CC 会自动 `atomicAdd` 扩展。

9.3 CC-Incremental 算法流程

```
// GPU 实现：和全量 CC 使用完全相同的 kernel!  
// 唯一区别是初始化：  
  
// 全量 CC：  
colors[] = -1;  
max_color = 4;  
  
// CC-Incremental:  
colors[] = prev_frame_colors[]; // 替代随机初始化  
max_color = prev_frame_max_color;  
  
// Round 1: 跳过着色 phase, 只做冲突检测  
// 大部分边：上一帧就不冲突，这一帧也不冲突 -> 跳过  
// 新增边 (u,v) 且 colors[u] == colors[v] -> 冲突!  
// loser 的颜色被清除 -> 加入 round 2 队列  
  
// Round 2+: 只对 losers 做正常 CC (快照 -> 选色 -> 冲突检测)  
// 通常只有极少数顶点，1-2 轮即可收敛
```

为什么 **CC** 比 **JP/CJP** 更适合做增量：

- JP/CJP 需要额外的 diff kernel 来找出受影响顶点，然后构建独立的重着色队列
- CC 不需要任何额外逻辑——它本来就对所有顶点做冲突检测，无冲突的自然跳过
- 相当于“零成本增量”：代码不变，只改初始状态

9.4 实验结果

单帧变化 (实验 2)

金字塔 15 body，删 6 边、增 3 边：

	CC 全量	CC-Incremental
颜色数	6	5
轮次	4	1
max_color	6	5

CC-Incremental 只需 1 轮（仅冲突检测），因为新增边没有恰好连接同色顶点。颜色数更少（ $5 < 6$ ），因为保留了上一帧的高质量着色。

多帧连续模拟 (实验 3)

8 帧 · 每帧随机增删 1-3 条边：

Frame	Del	Add	CC-Full(色/轮)	CC-Incr(色/轮)	新冲突
0	-	-	5/3	5/3	-
1	1	1	4/3	5/1	0
2	2	0	5/3	5/1	0
3	1	0	5/3	5/1	0
4	2	0	5/2	5/1	0
5	1	1	4/3	5/2	1
6	2	0	5/3	5/1	0
7	1	1	5/2	5/2	1

- 全量 CC 总轮次：19 · 增量 CC 总轮次：9 (节省 53%)
- 7 帧中只有 2 帧产生了新冲突 · 其余 5 帧只需 1 轮 (纯冲突检测即完成)
- 即使有冲突的帧也只需 2 轮 (检测 + 修复)

9.5 边界情况与策略

1. 大规模拓扑变化 (如场景切换)：大量新冲突 · CC-Incremental 退化为全量。解决方案：冲突率 > 50% 时直接全量。
2. 颜色漂移：max_color 只增不减。多帧后可能比最优值大。解决方案：每隔 N 帧 (如 60 帧) 做一次全量 CC · 重置 max_color = 4。
3. 删除边不减 max_color：图变稀疏后旧颜色号可能冗余，但保持不变是安全的。全量校准会自动修正。

9.5 GPU 实现路径

```
// 每帧着色流程
if (frame == 0 || force_full_recolor) {
    // 全量着色
    graph_coloring.color_jp(vtx_counts, vtx_table, n_bodies, stride);
} else {
    // 增量着色
    // 1. Diff kernel: 比较新旧邻接表, 找出 added/removed edges
    diff_edges_kernel<<<...>>>(
        prev_vtx_counts, prev_vtx_table,
        curr_vtx_counts, curr_vtx_table,
        d_added_edges, d_removed_edges, d_num_changes);

    // 2. Mark affected kernel: 标记受影响顶点
    mark_affected_kernel<<<...>>>(
        d_added_edges, d_num_added,
        d_colors, d_affected);

    // 3. Clear colors of affected vertices
```

```
clear_affected_kernel<<<...>>>(d_colors, d_affected, n_bodies);

// 4. JP recolor on affected vertices only
jp_recolor_kernel<<<...>>>(
    curr_vtx_counts, curr_vtx_table,
    d_colors, d_affected, n_bodies);
}

// 每 60 帧全量校准
if (frame % 60 == 0) force_full_recolor = true;
```

9.6 预期性能收益

场景	全量 JP 耗时	增量着色耗时	加速比
稳态 (无变化)	0.3-1.2 ms	~0.01 ms (仅 diff)	30-100x
轻微变化 (2-5 边)	0.3-1.2 ms	~0.05 ms	6-24x
中等变化 (10-20 边)	0.3-1.2 ms	~0.1-0.3 ms	3-4x
大规模变化 (>50% 边)	0.3-1.2 ms	~0.3-1.2 ms (退化)	1x

实际仿真中，大部分帧处于"稳态"或"轻微变化"，增量着色可以将着色开销从 ~1ms 降低到 <0.1ms，使图着色不再是 GPU solver 的瓶颈。

10. 算法选择总览

应用场景	推荐算法	增量着色	理由
实时仿真 (<500 body)	JP	可选	全量 JP 够快
中规模 (500-5K body)	CJP + 增量	必要	帧间相干性收益大
大规模 (>5K body)	CC + 增量	必要	最大化 GPU 利用率
调试/原型	Vivace	不需要	收敛快，适合验证
最少颜色	LDF	可选	颜色质量最优

11. 文件清单

文件	说明
src/rigid/avbd_cpu/avbd_graph_coloring.h	GraphColoringGPU 类声明
src/rigid/avbd_cpu/avbd_graph_coloring.cu	四种算法的 CUDA kernel + host 调度
src/rigid/avbd_cpu/avbd_solver.cpp	集成调用，每 300 帧打印着色对比
doc/graph_coloring_analysis.md	本文档
doc/graph_coloring_demo.py	Vivace/Luby/JP/LDF 四种算法的 Python 逐步演示

文件	说明
doc/graph_coloring_cjp_cc_demo.py	CJP/CC/增量着色 三种算法的 Python 逐步演示